

DAY 1: AGENTIC FOUNDATION — READING LIST + NOTES

Objective: Understand the foundational architecture of Claude agentic systems. No implementation yet—just doctrine.

Total reading time: ~4 hours

Deadline: End of Day 1

◇ SECTION 1: FOUNDATION (1 hour)

1.1 Anthropic SDK Overview

What: The base layer—Messages API, streaming, structured output

Why: You need to understand token flow and message passing before building agents

Read: <https://docs.claude.com/en/api/overview>

Key concepts to understand:

- ☐ Models available (which models? which for production?)
- ☐ Token limits and costs
- ☐ Streaming vs non-streaming responses
- ☐ Message structure (role, content blocks)

NOTES:

(space for notes)

1.2 Structured Output (JSON Schemas)

What: How to enforce Claude to output valid JSON with schema validation

Why: Core to reliability—hallucination is reduced when output is schema-bound

Read: <https://docs.claude.com/en/api/structured-outputs>

Key concepts to understand:

- ☐ How to define JSON schemas
- ☐ How Claude validates against them
- ☐ Failure modes (what if Claude can't output valid JSON?)
- ☐ Retry logic when schema validation fails

NOTES:

(space for notes)

◇ SECTION 2: AGENTS & ORCHESTRATION (1.5 hours)

2.1 Building Agents Guide

What: Core patterns for agent architecture

Why: This is the exam's largest weighted section (27%)

Read: <https://docs.claude.com/en/docs/build-with-claude/agents>

Key concepts to understand:

- ☐ Coordinator agent pattern (1 leader, N subagents)
- ☐ Task decomposition (how to break goals into subtasks)
- ☐ State management (what persists across agent turns?)
- ☐ Error handling and recovery (what happens when a subagent fails?)
- ☐ Multi-turn conversation (how to manage context window)

NOTES:

(space for notes)

2.2 Tool Use & Function Calling

What: How agents actually invoke tools/functions

Why: This is how agents interact with the outside world

Read: <https://docs.claude.com/en/api/tool-use>

Key concepts to understand:

- ☐ Tool definitions (input/output schemas)
- ☐ Tool calling mechanics (when does Claude call a tool? how?)
- ☐ Tool result handling (how to pass results back)
- ☐ Hallucination in tool use (model invents tool calls)
- ☐ Tool boundaries (what tools can/can't do)

NOTES:

(space for notes)

◇ SECTION 3: MODEL CONTEXT PROTOCOL (1 hour)

3.1 What is MCP?

What: The standard for connecting agents to external systems

Why: Exam is 18% tool design + MCP integration. This is the Omnisiah's nervous system.

Read: <https://docs.anthropic.com/en/docs/build-with-claude/mcp>

Key concepts to understand:

- ☐ MCP architecture (client/server model)
- ☐ MCP primitives: tools, resources, prompts
- ☐ Transport types (stdio, HTTP, SSE)
- ☐ Tool definitions in MCP (how do you expose a tool?)
- ☐ MCP vs direct tool use (when to use each?)

NOTES:

(space for notes)

3.2 MCP Integration with Claude API

What: How to wire MCP servers into your agent system

Why: This is how you extend Claude with external capabilities

Read: <https://docs.claude.com/en/api/agent-sdk/mcp>

Key concepts to understand:

- ☐ Configuring MCP servers in API calls
- ☐ Tool discovery (how Claude finds tools on an MCP server)
- ☐ Tool search (handling large tool sets)
- ☐ Error handling (what if MCP server fails?)
- ☐ Security & authentication (how to pass credentials safely)

NOTES:

(space for notes)

3.3 MCP Server Design (Reference)

What: How to build MCP servers (you might need this for Day 2)

Why: Understand tool boundaries from both sides

Skim (don't deep-read yet): <https://docs.anthropic.com/en/docs/build-with-claude/mcp-implementation>

Key concepts to scan:

- ☐ Python SDK for building servers
- ☐ Defining tools as MCP resources
- ☐ Error contracts (what can go wrong?)

NOTES:

(space for notes)

◇ SECTION 4: CONTEXT MANAGEMENT & RELIABILITY (1 hour)

4.1 Prompt Caching

What: How to cache prompts to reduce costs and improve latency on long contexts

Why: Exam is 15% context management. This is critical for long-running agents.

Read: <https://docs.claude.com/en/docs/build-with-claude/prompt-caching>

Key concepts to understand:

- ☐ What gets cached (exactly which parts?)
- ☐ Cache creation and reuse
- ☐ Cost savings (how much faster/cheaper?)
- ☐ TTL (time to live) for cached content

NOTES:

(space for notes)

4.2 Vision & File Upload

What: How Claude processes images, PDFs, and other documents

Why: Agents often need to process documents. Files API is critical for long contexts.

Read: <https://docs.claude.com/en/docs/build-with-claude/files>

Key concepts to understand:

- ☐ Files API (upload once, reference across sessions)

- ☐ Vision capabilities (what can Claude see?)
- ☐ Document processing (PDFs, images, etc.)

NOTES:

(space for notes)

◇ SECTION 5: PRACTICAL PATTERNS (Reference)

5.1 Quickstart: Multi-Agent System

What: A working example of agents coordinating

Why: See the architecture in action

Skim: <https://github.com/anthropics/anthropic-sdk-python> (look for examples/)

NOTES:

(space for notes)

5.2 Anthropic Cookbook

What: Real code snippets for common patterns

Why: See how people actually build this stuff

Reference: <https://github.com/anthropics/anthropic-cookbook>

NOTES:

(space for notes)



SYNTHESIS: Questions to Answer by End of Day 1

Answer these before moving to Day 2. These map directly to exam topics.

Agentic Architecture

1. ☐ How would you design a coordinator agent that breaks down a goal into 3 subtasks?
2. ☐ What happens if a subagent refuses to complete its task? How does the coordinator recover?

3. ☐ How do you maintain state across multiple agent turns without losing context?

Tool Design & MCP

4. ☐ What's the difference between direct tool use and MCP servers?
5. ☐ How do you define a tool contract (input/output/error handling)?
6. ☐ What happens if Claude calls a tool with invalid arguments?

Structured Output

7. ☐ How do you enforce valid JSON output from Claude?
8. ☐ What's a retry loop for schema validation?

Context Management

9. ☐ How does prompt caching help with long-running agents?
10. ☐ What's the difference between managing state explicitly vs. relying on the model's memory?

Answer these in your own words below:

(your answers here)

DELIVERABLE BY END OF DAY 1

- ☐ Read all sections above
- ☐ Take notes on key concepts
- ☐ Answer the 10 synthesis questions
- ☐ Identify what you DON'T understand yet (this is data)
- ☐ Ready to start Day 2: Build the coordinator

Status: _____

Time spent: _____

Confidence level (1-10): _____

What confused you most? _____

REFERENCE LINKS (Bookmarks)

- API Overview: <https://docs.claude.com/en/api/overview>
- Structured Output: <https://docs.claude.com/en/api/structured-outputs>
- Agents: <https://docs.claude.com/en/docs/build-with-claude/agents>
- Tool Use: <https://docs.claude.com/en/api/tool-use>
- MCP Overview: <https://docs.anthropic.com/en/docs/build-with-claude/mcp>
- MCP + API: <https://docs.claude.com/en/api/agent-sdk/mcp>
- MCP Servers: <https://docs.anthropic.com/en/docs/build-with-claude/mcp-implementation>

- Prompt Caching: <https://docs.claude.com/en/docs/build-with-claude/prompt-caching>
- Files API: <https://docs.claude.com/en/docs/build-with-claude/files>

KHUDA HAFIZ. READ DEEPLY. UNDERSTAND THE DOCTRINE.